# Frontend Recreation Plan

AI-Orchestration Content Factory

AI Agent Instruction Manual v5.0

---

Repository: github.com/HassanArif-collab/AI-Orchestration

Branch: codebase-audit-finding-fixes

This document is an instruction manual for an AI coding agent.

Each chunk must be completed and tested before the next begins.

Old frontend code is deleted only after the new build is verified.

April 2026

# Table of Contents


1. Overview 5

1.1 What You Are Building . . . 5

1.2 How Data Moves . . . 5

1.3 Build Strategy . . . 5

2. Pre-Work: Files the Agent Must Read 5

2.1 Backend API Routes (READ THESE) . . . 6

2.2 Frontend-to-Backend Connection Map . . . 6

2.3 Database Tables the Frontend Reads . . . 7

2.4 SSE Events (Two Separate Streams) . . . 7

Stream 1: Chat (POST /api/chat/stream) . . . 7

Stream 2: Pipeline Events (GET /api/events) . . . 7

3. Chunk 0: Project Foundation 8

3.1 Files to Read . . . 8

3.2 What to Create . . . 8

3.3 Mistakes to Avoid . . . 9

3.4 How to Test . . . 9

4. Chunk 1: Types and API Client 9

4.1 Files to Read . . . 9

4.2 What to Create . . . 9

4.3 Mistakes to Avoid . . . 10

4.4 How to Test . . . 10

5. Chunk 2: Layout Shell 11

5.1 Files to Read . . . 11

5.2 What to Create . . . . . 11
5.3 Mistakes to Avoid . . . . . 11
5.4 How to Test . . . . . 11
6. Chunk 3: Kanban Board Core 11
6.1 Files to Read . . . . . 12
6.2 What to Create . . . . . 12
6.3 Mistakes to Avoid . . . . . 12
6.4 How to Test . . . . . 12
7. Chunk 4: Card Details and Drawer 13
7.1 Files to Read . . . . . 13
7.2 What to Create . . . . . 13
7.3 Mistakes to Avoid . . . . . 13
7.4 How to Test . . . . . 14
8. Chunk 5: Review and Approval Flow 14
8.1 Files to Read . . . . . 14
8.2 What to Create . . . . . 14
8.3 Mistakes to Avoid . . . . . 14
8.4 How to Test . . . . . 14
9. Chunk 6: Chat Panel 15
9.1 Files to Read . . . . . 15
9.2 What to Create . . . . . 15
9.3 Mistakes to Avoid . . . . . 15
9.4 How to Test . . . . . 15
10. Chunk 7: Sidebar Panels 15
10.1 Files to Read . . . . . 15

10.2 What to Create . . . . . 16
10.3 Mistakes to Avoid . . . . . 16
10.4 How to Test . . . . . 16
11. Chunk 8: Telemetry and Optimizations 16
11.1 What to Create . . . . . 16
11.2 Mistakes to Avoid . . . . . 17
11.3 How to Test . . . . . 17
12. Chunk 9: Responsive Polish 17
12.1 What to Create . . . . . 17
12.2 How to Test . . . . . 17
13. Chunk 10: Final Verification and Old Code Deletion 17
13.1 Final Verification Checklist . . . . . 17
13.2 Delete Old Code . . . . . 18
13.3 What Gets Deleted . . . . . 18
14. Quick Reference: Chunk Dependency Map 18
15. Risk Notes 19

# 1. Overview

This is an instruction manual for an AI coding agent to rebuild the frontend of the AI-Orchestration Content Factory from the ground up. The agent will work through 10 sequential chunks, each building on the previous one. Every chunk is independently testable. The old frontend code is deleted only after the entire new build is verified and working.

## 1.1 What You Are Building

A Kanban-based dashboard for managing an AI-powered YouTube content pipeline. The dashboard has three main areas: (1) a Header with a topic discovery input, (2) a central Kanban Board with 6 columns containing draggable cards, and (3) a right-side Sidebar with 6 tabbed panels (Chat, YouTube, Quota, Models, Skills, Knowledge Base). A CardDrawer overlay slides in when any card is clicked.

- Header: Text input to trigger topic discovery pipeline
- Board: 6 columns (Topic Finding, Suggested Topics, Researching, Script Evolution, Review + Visuals, Published)
- Sidebar: 6 tabs - Chat (SSE streaming), YouTube (analytics), Quota (provider limits), Models (agent assignments), Skills (pipeline config), Knowledge Base
- CardDrawer: Slide-in panel showing topic brief, agent activity log, script preview, review/approval

## 1.2 How Data Moves

The frontend communicates with the backend through three channels:

- Supabase Realtime (WebSocket) - Kanban cards and agent thoughts update live. The frontend subscribes to INSERT/UPDATE/DELETE on the kanban_cards table and INSERT on agent_thoughts table. SWR caches the data and auto-refreshes on Realtime events.
- REST API (14 endpoints) - Pipeline actions (discover, produce, resume), card management (save, delete, move), analytics, quota, skills, knowledge base. All require X-API-Key header because API_AUTH_ENABLED defaults to TRUE.
- SSE (Server-Sent Events) - Two separate streams: (1) Chat streaming via POST to /api/chat/stream returning token/tool_start/tool_end/done/error events, and (2) Pipeline events via GET /api/events returning pipeline_update/stage_complete/human_gate/pipeline_complete/iteration_complete events.

## 1.3 Build Strategy

Build the new frontend in a separate directory (apps/web-v2/) alongside the old one (apps/web/). This lets you test the new build against the running backend without breaking the old UI. After all 10 chunks are complete and verified, delete the old apps/web/ directory and rename apps/web-v2/ to apps/web/.

- Chunk 0-1: Project setup, types, API client, SSE client (no visible UI yet)
- Chunk 2-3: Layout shell, Kanban board with drag-and-drop (core visual)
- Chunk 4-5: Card drawer, review/approval flow (detail views)
- Chunk 6-7: Chat panel with SSE streaming, remaining sidebar panels
- Chunk 8-9: Telemetry optimizations, responsive polish
- Chunk 10: Final verification, delete old code, rename to apps/web/

# 2. Pre-Work: Files the Agent Must Read

Before writing any code, read these files to understand the backend contracts, data shapes, and how the existing frontend works. Every connection point between frontend and backend is documented here.

## 2.1 Backend API Routes (READ THESE)

The backend has 75 API endpoints across 11 route modules. The frontend uses 14 of them directly. Read the route files to understand request/response shapes:

- apps/api/routes/pipeline_routes.py - discover, produce, langgraph/state, langgraph/resume, langgraph/preview
- apps/api/routes/kanban_routes.py - tasks CRUD, cards/save, tasks/extend, cards/move
- apps/api/routes/chat_routes.py - chat/stream (SSE), chat/message, chat/history
- apps/api/routes/analytics_routes.py - competitors, channel, repurpose
- apps/api/routes/provider_routes.py - quota (GET /api/providers/quota)
- apps/api/routes/settings_routes.py - skills (GET /api/settings/skills), knowledge-base

## 2.2 Frontend-to-Backend Connection Map

This table shows every connection point. The agent MUST implement all of these. None can be skipped.

| Frontend Action | Method | Endpoint / Source | Purpose |
|---|---|---|---|
| Header input | POST | /api/pipeline/discover | Start topic discovery |
| Column 2 card action | POST | /api/kanban/cards/{id}/save | Save topic (remove 3h expiry) |
| Column 2 card action | POST | /api/kanban/tasks/{id}/extend | Extend expiry by 3 hours |
| Column 3 card action | POST | /api/pipeline/produce/{id} | Start production pipeline |
| Card drag-and-drop | PATCH | /api/kanban/tasks/{id} | Move card (body: {stage: col_num}) |
| Card delete button | DELETE | /api/kanban/tasks/{id} | Delete card |
| CardDrawer (any card) | GET | /api/pipeline/langgraph/state/{id} | Poll pipeline state every 5s |
| CardDrawer (col 5) | POST | /api/pipeline/langgraph/resume/{id} | Approve/reject script |
| CardDrawer (col 5) | GET | /api/pipeline/langgraph/preview/{id} | Preview before publish |
| ChatPanel send | POST | /api/chat/stream | SSE chat stream (POST, not GET!) |
| ChatPanel history | GET | /api/chat/history/{session_id} | Load past conversation |
| Sidebar YouTube tab | GET | /api/analytics/competitors | Competitor video analytics |
| Sidebar YouTube tab | GET | /api/analytics/channel | Own channel stats |
| Sidebar YouTube tab | POST | /api/analytics/repurpose | Repurpose competitor video |
| Sidebar Quota tab | GET | /api/providers/quota | Provider rate limits (poll 5s) |

| Frontend Action | Method | Endpoint / Source | Purpose |
|---|---|---|---|
| Sidebar Models tab | GET | /api/settings/skills | Agent-to-model assignments |
| Sidebar Skills tab | GET | /api/settings/skills | Skill file contents |
| Sidebar KB tab | GET | /api/settings/knowledge-base | Knowledge base content |
| Supabase Realtime | WS | kanban_cards table | Subscribe to all CRUD events |
| Supabase Realtime | WS | agent_thoughts table | Subscribe to INSERT events |
| SSE EventSource | GET | /api/events | Pipeline events (8 types) |

Table 1. Complete Frontend-to-Backend Connection Map (all must be implemented)

## 2.3 Database Tables the Frontend Reads

The frontend does NOT query the database directly. It reads through two channels: (1) Supabase Realtime subscriptions for live updates, and (2) REST API endpoints for specific queries. However, the TypeScript types MUST match the database schema exactly.

- kanban_cards - Primary table. Columns: id (UUID), title (TEXT), column_index (INT 1-6), pipeline_run_id (TEXT nullable), parent_id (UUID nullable), color (TEXT), metadata (JSONB - contains topic_brief and viability_score), status (TEXT), position (INT), expires_at (TIMESTAMPTZ), created_at, updated_at
- agent_thoughts - Activity log per card. Columns: id, card_id (FK CASCADE), agent_name (TEXT), thought_type (TEXT: thinking/search/output/error/memory_read/memory_write), content (TEXT), created_at
- pipeline_runs - Pipeline execution state. Columns: run_id (TEXT PK), current_stage, stage_outputs (JSONB), stage_status (JSONB), status, error_message

INFO: topic_brief and viability_score are stored INSIDE the metadata JSONB column, NOT as separate top-level columns. Extract them from card.metadata.topic_brief and card.metadata.viability_score.

## 2.4 SSE Events (Two Separate Streams)

There are TWO independent SSE streams. The agent must implement both:

### Stream 1: Chat (POST /api/chat/stream)

Initiated via POST (not GET, because it requires a JSON body). Parse the response as ReadableStream. Each line is "data: {JSON}" followed by newline. Handle these event types:

| Event Type | Frontend Action |
|---|---|
| token | Append content to streaming text buffer |
| tool_start | Show "Calling {tool}..." indicator, add to active tools list |
| tool_end | Remove tool from active tools list |
| done | Finalize message. Save session_id from response for conversation continuity |
| error | Show error toast, append error text to chat |

Table 2. Chat SSE Events

CRITICAL: There is NO "progress" event in the chat stream. Pipeline progress comes from the second SSE stream (GET /api/events) and Supabase Realtime.

### Stream 2: Pipeline Events (GET /api/events)

Use EventSource API (GET request, no body needed). Listen for named events:

| Event | Key Payload Fields | Frontend Action |
|---|---|---|
| pipeline_update | run_id, stage, status | Update pipeline progress in CardDrawer |
| stage_complete | run_id, stage, summary | Show stage completion indicator |
| human_gate | run_id, stage | Alert user that review is needed (Column 5) |
| pipeline_complete | run_id | Mark pipeline as complete, refresh card |
| iteration_complete | run_id, iteration, score, mutation_zone, beat_baseline | Update iteration counter and score display |
| provider_status | provider, status | Update provider status indicator |
| task_created | task_data (full card dict) | Refresh kanban board with new card |
| rate_limit | wait_time, attempt, model | Show rate limit warning toast |

Table 3. Pipeline SSE Events (GET /api/events via EventSource)

# 3. Chunk 0: Project Foundation

Goal: Create the project scaffolding in apps/web-v2/ with all dependencies, build configuration, and base infrastructure. No visible UI yet.

## 3.1 Files to Read

- apps/web/package.json - See current dependency versions
- apps/web/vite.config.ts - See current Vite configuration
- apps/web/tsconfig.json - See TypeScript configuration
- apps/web/src/index.css - See Tailwind v4 setup and custom styles

## 3.2 What to Create

- apps/web-v2/ directory with:
  - package.json - Install these exact versions: react ^19.2.4, react-dom ^19.2.4, typescript ^5.9.3, vite ^8.0.1, @vitejs/plugin-react ^4.3.0, tailwindcss ^4.2.2, @tailwindcss/vite ^4.0.0, swr ^2.4.1, @supabase/supabase-js ^2.101.0, @dnd-kit/react ^1.0.0 (NOT @dnd-kit/core), zustand ^5.0.0, zod ^3.23.0, react-markdown ^10.1.0, remark-gfm ^4.0.0, rehype-sanitize ^6.0.0, lucide-react ^0.460.0, clsx ^2.1.0, date-fns ^3.6.0, react-error-boundary ^4.0.0, @radix-ui/react-dialog ^1.1.0, @radix-ui/react-tabs ^1.1.0, @tanstack/react-virtual ^3.10.0
  - vite.config.ts - React plugin + Tailwind plugin + path alias (@/ -> src/) + API proxy (/api -> localhost:3000)
  - tsconfig.json - Strict mode, ES2022 target, path aliases
  - .env.example - Placeholders ONLY: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_KEY, VITE_API_URL
  - src/main.tsx - Entry point, wrap App in ErrorBoundary
  - src/App.tsx - Root component, renders MainLayout
  - src/index.css - Tailwind v4 import (@import "tailwindcss"), custom theme colors
  - src/lib/store.ts - Zustand store for UI state: sidebarOpen (bool), activeTab (string), selectedCardId (string|null), drawerOpen (bool), filters ({column, search}). Use selector-based pattern to prevent re-renders.
  - src/lib/config.ts - Centralized constants: API_BASE_URL, DEFAULT_TIMEOUT (15000), MAX_RETRIES (2), POLL_INTERVALS (pipeline: 5s, quota: 5s, youtube: 5min), COLUMN_NAMES (1-6 mapping), MAX_TOASTS (5)
  - src/components/ui/ErrorBoundary.tsx - 3-tier strategy: Tier 1 (root) shows full-page reload, Tier 2 (feature sections) shows section retry, Tier 3 (widgets) shows inline error. Use react-error-boundary library.

- src/hooks/useToast.ts - Zustand-based toast system. Max 5 concurrent. Auto-dismiss after 5s. Export showToast() function. Use crypto.randomUUID() for toast IDs.

### 3.3 Mistakes to Avoid

~~DO NOT: Do NOT install @dnd-kit/core or @dnd-kit/sortable. They are deprecated. Use @dnd-kit/react only.~~
~~DO NOT: Do NOT put real Supabase credentials in .env.example. Use placeholders like YOUR_SUPABASE_URL_HERE.~~
~~DO NOT: Do NOT use React Context for UI state. Use Zustand. Context causes unnecessary re-renders across the tree.~~
~~DO NOT: Do NOT skip the API proxy in vite.config.ts. Without it, you get CORS errors on every /api call during development.~~

### 3.4 How to Test

- npm run dev starts without errors at localhost:5173
- Navigate to /api/health - should return healthy (proxied to backend)
- tsc --noEmit passes with zero errors
- Open browser console - no errors

## 4. Chunk 1: Types and API Client

Goal: Create the complete type system and API client layer. All components in later chunks depend on these types. This chunk has no visible UI.

### 4.1 Files to Read

- apps/web/src/types/index.ts - Current type definitions (has bugs to avoid)
- apps/web/src/lib/api.ts - Current API client (reference for endpoints)
- apps/web/src/lib/supabase.ts - Current Supabase config
- apps/web/src/lib/errorMapper.ts - Current error mapping
- apps/web/src/lib/cardHelpers.ts - Current card action logic
- apps/api/routes/ - All route files for request/response shapes
- supabase/migrations/001_initial_schema.sql - Database schema

### 4.2 What to Create

- src/types/index.ts - Domain types using Zod for runtime validation:
  - KanbanCard type: MUST use column_index (not "column") to match DB. MUST include parent_id (string|null). MUST include metadata (Record). MUST include pipeline_run_id. topic_brief is inside metadata, not top-level. Add helper functions getTopicBrief(card) and getViabilityScore(card) that extract from metadata.
  - AgentThought type: id, card_id, agent_name, thought_type (enum: thinking/search/output/error/memory_read/memory_write), content, created_at
  - PipelineStateResponse type: card_id, values: {evaluation_score, best_score, iteration_count, pipeline_status, current_draft, visual_plan, error}, next: string[]
  - Column definitions: COLUMNS record mapping 1-6 to {name, description, color}
  - PipelineStatus type: discovering, grading, suggested, researching, drafting, scoring, mutating, visuals, review, publishing, complete, error
- src/types/api.ts - Zod schemas for all 14 API request/response types
- src/lib/api.ts - HTTP client:
  - Always send X-API-Key header (API_AUTH_ENABLED defaults TRUE)

- Timeout: 15s default via AbortController
- Retry: max 2 retries on 408/429/5xx with exponential backoff (1s, 2s)
- JSON parse: wrap in try/catch, throw user-friendly error on failure
- Define ALL 14 endpoints as typed methods on api object (discover, produce, resume, getPipelineState, getQuota, getSkills, saveCard, deleteCard, moveCard, getCompetitors, getOwnChannel, repurpose, getKnowledgeBase, getChatHistory)
- IMPORTANT: moveCard uses PATCH /api/kanban/tasks/{id} with body {stage: columnNumber}, NOT PUT /api/kanban/cards/{id}/move
- IMPORTANT: deleteCard uses DELETE /api/kanban/tasks/{id}, NOT /api/kanban/cards/{id}

• src/lib/sse.ts - Reusable SSE client class:
- Support POST-based streaming (fetch + ReadableStream, NOT EventSource - EventSource is GET only)
- Mutex: only one stream at a time (isSending flag)
- Handle all 5 chat event types: token, tool_start, tool_end, done, error
- Auto-cleanup via AbortController on abort()
- Buffer incomplete lines across chunks (split on \n, keep last partial line)

• src/lib/pipelineEvents.ts - Pipeline SSE client (uses EventSource for GET):
- Connect to GET /api/events
- Listen for 8 named events: pipeline_update, stage_complete, human_gate, pipeline_complete, iteration_complete, provider_status, task_created, rate_limit
- Auto-reconnect on disconnect
- Callback-based: onPipelineUpdate, onStageComplete, onHumanGate, etc.

• src/lib/supabase.ts - Same pattern as current: nullable client, isSupabaseConfigured() check

• src/lib/errorMapper.ts - Map API errors to user-friendly messages. Handle: 400, 401, 403, 404, 408, 409, 422, 429, 500+

• src/lib/cardHelpers.ts - Card action logic per column: Column 1 (no actions, discovering), Column 2 (save topic, extend expiry, start production), Column 3-4 (read-only, pipeline running), Column 5 (approve/reject), Column 6 (read-only, published)

## 4.3 Mistakes to Avoid

CRITICAL: Do NOT use "column" as the field name in KanbanCard. The DB column is "column_index". Using "column" causes silent data mismatches.
DO NOT: Do NOT make topic_brief and viability_score top-level fields on KanbanCard. They live inside the metadata JSONB column. The old code had them as top-level fields which do not exist in the database.
DO NOT: Do NOT skip parent_id on KanbanCard. The old code was missing this field entirely.
DO NOT: Do NOT use the same endpoint naming for save vs move. saveCard is POST /api/kanban/cards/{id}/save. moveCard is PATCH /api/kanban/tasks/{id} with {stage: col}. Note the different URL patterns (cards vs tasks).
DO NOT: Do NOT use EventSource for chat streaming. Chat requires POST with a body. Use fetch + ReadableStream.
DO NOT: Do NOT handle a "progress" event in chat SSE. It does not exist. Pipeline progress comes from GET /api/events.

## 4.4 How to Test

• Import all types - no TypeScript errors
• Call api.getQuota() with backend running - should return provider data
• Call api.getPipelineState("test-id") - should return error (expected, no such card)
• Zod schemas: parse a mock KanbanCard object - should pass validation
• SSE client: create instance, call abort() - no errors

# 5. Chunk 2: Layout Shell

Goal: Build the visual skeleton - MainLayout, Header, Sidebar with tabs. The Sidebar is the most critical piece here because of the tab-switching bug in the old code.

## 5.1 Files to Read

- apps/web/src/layout/MainLayout.tsx - Current layout structure
- apps/web/src/layout/Header.tsx - Current header with discover input
- apps/web/src/layout/Sidebar.tsx - CRITICAL READ: understand the tab-switching bug
- apps/web/src/components/ui/ToastContainer.tsx - Toast rendering

## 5.2 What to Create

- src/layout/MainLayout.tsx - Horizontal flex: Header (top, full width) + div (flex row) containing Board (flex-1) and Sidebar (w-96, collapsible). ToastContainer as fixed overlay.
- src/layout/Header.tsx - App title on left, topic discovery input + "Discover" button on right. On submit: call api.discover({seed_hint: value}), show loading state, show success/error toast. Disable input while loading.
- src/layout/Sidebar.tsx - 6 tabs with icons from lucide-react (not emoji). Tabs: Chat, YouTube, Quota, Models, Skills, KB. Collapsible via toggle button.
- src/components/ui/ToastContainer.tsx - Renders toasts from Zustand store. Fixed position bottom-right. Auto-dismiss animation. Max 5 visible.

## 5.3 Mistakes to Avoid

CRITICAL: Do NOT use conditional rendering for tab content (e.g., {activeTab === "chat" && }). This DESTROYS the ChatPanel component on every tab switch, killing the SSE connection and losing all chat history. The old code did exactly this. Instead, use CSS display:none/block to hide/show panels without unmounting them.

DO NOT: Do NOT use emoji for tab icons (the old code used emojis). Use lucide-react icons: MessageSquare, Youtube, BarChart3, Bot, FileText, BookOpen.

DO NOT: Do NOT set sidebarOpen only once on mount with window.innerWidth. The old code did this and never responded to resize events. Use a resize event listener or set initial state from window width in Zustand store.

DO NOT: Do NOT skip aria attributes on tabs. Use role="tablist", role="tab", aria-selected, aria-controls, tabIndex for keyboard navigation.

## 5.4 How to Test

- Page renders with Header, empty Board area, and Sidebar
- Click sidebar toggle - sidebar opens/closes
- Click each tab - tab highlights change
- Type in Header input and press Enter - loading state shows, API call fires (check Network tab)
- Resize window below 1024px - sidebar starts collapsed
- Verify: when you switch away from Chat tab and back, the Chat component is NOT remounted (check React DevTools)

# 6. Chunk 3: Kanban Board Core

Goal: Build the Kanban board with 6 columns, draggable cards, and real-time updates. This is the most visually complex chunk.

## 6.1 Files to Read

- apps/web/src/components/kanban/Board.tsx - Current board with DnD
- apps/web/src/components/kanban/Column.tsx - Column rendering
- apps/web/src/components/kanban/Card.tsx - Card component
- apps/web/src/hooks/useCards.ts - Supabase Realtime subscription pattern
- apps/web/src/hooks/useCardTimer.ts - Expiration countdown
- apps/web/src/components/common/StatusBadge.tsx, EmptyState.tsx, ProgressBar.tsx
  - @dnd-kit/react documentation - NEW API, different from @dnd-kit/core

## 6.2 What to Create

- src/hooks/useCards.ts - Fetch all kanban_cards from Supabase (SELECT * ORDER BY created_at DESC). Subscribe to Realtime postgres_changes on kanban_cards table (event: *). On any change, call SWR mutate() to refetch. Export groupByColumn helper using useMemo.
- src/hooks/useAgentStream.ts - Subscribe to Realtime INSERT on agent_thoughts for a specific card_id. Append new thoughts to in-memory list. Load history on mount (query by card_id ORDER BY created_at ASC). Clean up subscription on unmount.
- src/hooks/useCardTimer.ts - Countdown timer for cards in Column 2 with expires_at. Only run timer for cards in column_index 2 (do NOT run for all cards). Show remaining time, change color when < 30 minutes.
- src/components/kanban/Board.tsx - 6 columns in horizontal scroll. DndContext with closestCenter collision detection. DragOverlay with floating card preview. Optimistic updates on drag: update local state immediately, call API, revert on failure. Transition validation: cards can only move forward (1->2, 2->3, etc.), show toast on invalid move.
- src/components/kanban/Column.tsx - Fixed width (280px). Column header with name, description, card count badge. Scrollable card list. Drop target using @dnd-kit/react droppable. EmptyState when no cards.
- src/components/kanban/Card.tsx - Draggable item. Shows: topic brief title (from metadata), description (line-clamp-2), status badge, viability score (from metadata), expiration timer (Column 2 only). Action buttons based on column: save/extend/produce (col 2). Click opens CardDrawer.
- src/components/common/StatusBadge.tsx - Color-coded status indicator. idle=gray, thinking/discovering=blue, researching=green, scoring/drafting=purple, review=orange, complete=emerald, error=red.
- src/components/common/EmptyState.tsx - Centered text with icon when column has no cards.
- src/components/common/ProgressBar.tsx - Pipeline stage progress bar for CardDrawer. Shows current stage name, iteration count, best score.

## 6.3 Mistakes to Avoid

CRITICAL: Do NOT use non-null assertion (!) on cards.find() for DragOverlay. If a card is deleted between drag start and overlay render, the app crashes. Use optional chaining and a null check with a fallback.

DO NOT: Do NOT use @dnd-kit/core or @dnd-kit/sortable imports. The new package is @dnd-kit/react and has a different API surface. Read the @dnd-kit/react docs.

DO NOT: Do NOT recalculate grouped cards on every render. Use useMemo with cards as dependency.

DO NOT: Do NOT run the expiration timer for every card on the board. Only run it for cards in column_index 2. The old code ran timers for all cards, wasting CPU.

DO NOT: Do NOT allow backward card moves via drag. Only forward transitions (1->2, 2->3, etc.) are valid. The old code allowed any drag direction.

DO NOT: Do NOT forget to call mutate() after successful card save/delete. The old code did not refresh the card list after these actions, leaving stale UI.

## 6.4 How to Test

• With backend running: board loads cards from Supabase, grouped into 6 columns

• Drag a card from Column 2 to Column 3 - card moves, API call succeeds, card list refreshes

• Try dragging backward (Column 3 to Column 2) - toast shows "invalid move"

• Click a card - CardDrawer opens (empty for now, next chunk)

• Delete a card via action button - card disappears from board

• Start topic discovery from Header - new card appears in Column 1, moves to Column 2 when done

• Open browser in two tabs - create card in one tab, appears in the other via Realtime

# 7. Chunk 4: Card Details and Drawer

Goal: Build the CardDrawer that slides in when a card is clicked. Shows topic brief, agent activity log, pipeline progress, and script preview.

## 7.1 Files to Read

• apps/web/src/components/kanban/CardDrawer.tsx - Current drawer

• apps/web/src/components/common/AgentLog.tsx - Agent thought display

• apps/web/src/hooks/usePipelineState.ts - Pipeline state polling

- packages/content_factory/orchestration/nodes.py - Understand what each pipeline stage does

- packages/content_factory/orchestration/state.py - LangGraph state fields

## 7.2 What to Create

• src/hooks/usePipelineState.ts - Poll GET /api/pipeline/langgraph/state/{id} every 5 seconds via SWR. Stop polling when pipeline_status is "complete" or "error". Return: pipeline state data, current stage, score, iteration count, draft text.

• src/hooks/usePipelineEvents.ts - Connect to GET /api/events via EventSource. Listen for pipeline_update, stage_complete, human_gate, pipeline_complete, iteration_complete. Trigger SWR mutate on pipeline state when these events arrive.

• src/components/kanban/CardDrawer.tsx - Slide-in panel from right side (600px width). Use @radix-ui/react-dialog for accessibility (focus trap, Escape to close, screen reader announcements). Sections:

- Header: Card title + status badge + close button

- Pipeline Progress: Stage name, progress bar, iteration count, current score

- Agent Activity Log: List of agent thoughts from useAgentStream, color-coded by thought_type (blue=thinking, green=search, purple=output, red=error, yellow=memory_read/write). Auto-scroll to bottom on new thoughts.

- Script Preview: Show current_draft from pipeline state. Render as markdown. Show visual_plan annotations separately.

- Card Actions: Context-dependent buttons (save, extend, produce, approve/reject)

- 500ms delay before closing after approve/reject so user sees the result

• src/components/common/AgentLog.tsx - Scrollable list of agent thoughts. Each entry shows: agent_name, thought_type icon/color, content text, timestamp. Use @tanstack/react-virtual for virtualization if > 50 thoughts.

## 7.3 Mistakes to Avoid

CRITICAL: Do NOT use custom querySelector("button[onClick]") for focus trapping. The old code did this and it is unreliable with React synthetic events. Use @radix-ui/react-dialog which handles focus trapping automatically.

DO NOT: Do NOT close the drawer immediately after approve/reject. The old code closed instantly so the user never saw the result. Add a 500ms delay.

DO NOT: Do NOT poll pipeline state when the pipeline is complete or errored. The old code polled every 5 seconds forever, even for finished pipelines.

DO NOT: Do NOT use lg:grid-cols-2 for ScriptViewer inside the 600px drawer. lg breakpoint is 1024px which never triggers inside a 600px container. Use responsive breakpoints appropriate for the drawer width.

### 7.4 How to Test

- Click a card in Column 1 (discovering) - drawer opens, shows agent thoughts streaming in
- Click a card in Column 5 (review) - drawer opens, shows script preview + approve/reject buttons
- Press Escape - drawer closes
- Start a discovery pipeline - watch agent thoughts appear in real-time in the drawer
- Verify pipeline progress bar updates as stages complete

## 8. Chunk 5: Review and Approval Flow

### 8.1 Files to Read

- apps/web/src/components/review/ReviewPanel.tsx - Current review UI
- apps/web/src/components/review/ScriptViewer.tsx - Script display
- apps/web/src/components/review/PublishConfirmModal.tsx - Publish confirmation

### 8.2 What to Create

- src/components/review/ReviewPanel.tsx - Appears only for cards in Column 5. Contains: feedback textarea (optional), Reject button (red), Approve & Publish button (green). On submit: call api.resume(cardId, {approved, feedback}). Disable both buttons while submitting. Add AbortController to cancel if API hangs.
- src/components/review/ScriptViewer.tsx - Display the current draft with markdown rendering. Show visual_plan annotations (B-ROLL, MAP, DATA markers) as highlighted inline badges. Side-by-side layout at wider widths, stacked at narrow widths (use the drawer width as reference, not viewport width).
- src/components/review/PublishConfirmModal.tsx - Confirmation dialog before publish. Show a countdown (3 seconds) before the publish button becomes active. Disable after first click to prevent double-publish. Show success/error result.

### 8.3 Mistakes to Avoid

DO NOT: Do NOT allow double-publish. The old code had no prevention. Disable the button after first click and show a loading state.

DO NOT: Do NOT have both Approve and Reject buttons disabled with no way to cancel a hanging API call. Add an AbortController and a Cancel button.

DO NOT: Do NOT close the CardDrawer before the user sees the approve/reject result. The parent component handles the 500ms delay, but also show a visual confirmation in the ReviewPanel itself.

### 8.4 How to Test

- Open a card in Column 5 - review panel shows with Approve and Reject buttons
- Type feedback and click Approve - API call fires, buttons disable, result shows
- Click Reject with no feedback - should still work (feedback is optional)
- Verify publish confirmation modal appears before final publish

# 9. Chunk 6: Chat Panel

Goal: Build the streaming chat panel with SSE, multi-turn conversation, and XSS-safe markdown rendering.

## 9.1 Files to Read

- apps/web/src/components/chat/ChatPanel.tsx - Current chat implementation
- apps/web/src/components/chat/ChatMessage.tsx - Message rendering (HAS XSS BUG)
- apps/web/src/hooks/useChat.ts - SSE streaming pattern
- apps/api/routes/chat_routes.py - Chat stream format

## 9.2 What to Create

- src/hooks/useChat.ts - Manage message list (user + assistant messages). Use SSEClient from src/lib/sse.ts. On send: add user message to list, create assistant placeholder, stream tokens into it. Store session_id for conversation continuity. Mutex: prevent concurrent sendMessage calls. On unmount: abort SSE connection.
- src/components/chat/ChatMessage.tsx - Render a single message. User messages: simple text, right-aligned, blue background. Assistant messages: markdown rendered via react-markdown + remark-gfm + rehype-sanitize (CRITICAL for XSS prevention). Wrap in React.memo to prevent re-renders when other messages receive tokens. Show streaming cursor animation on the last message while streaming.
- src/components/chat/ChatPanel.tsx - Message list (scrollable, auto-scroll to bottom on new messages), input field + send button, tool call indicator bar ("Calling research..."), loading state. Send on Enter key or button click. Clear input after send. Load chat history on mount if session_id exists.

## 9.3 Mistakes to Avoid

CRITICAL: Do NOT render API markdown responses without rehype-sanitize. The old code had an XSS vulnerability - API responses could inject arbitrary HTML/JavaScript. Always use rehypePlugins={[rehypeSanitize]} on every ReactMarkdown instance.

~~CRITICAL: Do NOT re-render the entire message list on every streaming token. The old code did this, causing severe lag. Wrap ChatMessage in React.memo so only the currently streaming message re-renders.~~

~~DO NOT: Do NOT allow concurrent sendMessage calls. The old code had a race condition where multiple SSE connections competed. Use a ref-based mutex (isSending flag).~~

~~DO NOT: Do NOT destroy the SSE connection when switching sidebar tabs. This is already handled by the CSS~~ display:none approach in the Sidebar (Chunk 2), but also add cleanup on component unmount.

## 9.4 How to Test

- Send a message - tokens stream in real-time
- Send multiple messages - conversation context is maintained (session_id persists)
- Switch to another tab and back - chat history is preserved, SSE connection still active
- Kill the backend during streaming - error appears, connection can recover
- Send a message containing HTML tags - verify they are sanitized (no XSS)

# 10. Chunk 7: Sidebar Panels

## 10.1 Files to Read

- apps/web/src/components/youtube/YouTubePanel.tsx, CompetitorCard.tsx, OwnStats.tsx
- apps/web/src/components/telemetry/QuotaPanel.tsx, ModelRegistry.tsx

• apps/web/src/components/system/SkillViewer.tsx, KnowledgeBase.tsx

### 10.2 What to Create

• src/components/youtube/YouTubePanel.tsx - Two sub-tabs: "Competitors" and "Own Channel". Competitor tab fetches GET /api/analytics/competitors and renders CompetitorCard list. Own Channel tab fetches GET /api/analytics/channel and shows subscriber count, total views, video count.

• src/components/youtube/CompetitorCard.tsx - Video thumbnail with lazy loading (loading="lazy"), onError fallback, aspect-ratio CSS. Title, channel name, view count, and "Repurpose" button that calls POST /api/analytics/repurpose.

• src/components/youtube/OwnStats.tsx - Three stat cards: subscribers, total views, video count. Fetched from GET /api/analytics/channel.

• src/components/telemetry/QuotaPanel.tsx - Table of provider rate limits from GET /api/providers/quota. Poll every 5 seconds. Show provider name, RPM remaining, TPM remaining, last updated time. Color-code: green (healthy), yellow (< 50%), red (< 10%).

• src/components/telemetry/ModelRegistry.tsx - Table of agent-to-model assignments from GET /api/settings/skills. Show agent name, model, provider.

• src/components/system/SkillViewer.tsx - Display skill file contents from GET /api/settings/skills. Show file name as tab, content as code block or formatted text.

• src/components/system/KnowledgeBase.tsx - Display knowledge base content from GET /api/settings/knowledge-base. Render as markdown with rehype-sanitize (same XSS prevention as ChatMessage).

### 10.3 Mistakes to Avoid

DO NOT: Do NOT fetch competitor videos when the "Own Channel" sub-tab is active. The old code always fetched competitors regardless of which tab was visible. Only fetch when the tab is actually shown.
DO NOT: Do NOT skip loading="lazy" and onError on competitor thumbnails. The old code had broken image display.
DO NOT: Do NOT render KnowledgeBase markdown without rehype-sanitize. Same XSS risk as ChatMessage.
DO NOT: Do NOT hardcode the ModelRegistry data. The old code had a static MODEL_REGISTRY array. Fetch it from the API.

### 10.4 How to Test

• Click YouTube tab - competitor videos load with thumbnails

• Switch to Own Channel sub-tab - channel stats display

• Click Quota tab - provider rate limits show and refresh every 5s

• Click Repurpose on a competitor video - card is created (check Board)

• Click Knowledge Base tab - content renders as markdown

## 11. Chunk 8: Telemetry and Optimizations

### 11.1 What to Create

• Centralize all retry/timeout/backoff values in src/lib/config.ts (the old code scattered these across 4+ files)

• Stop pipeline polling when status is "complete" or "error" (the old code polled forever)

• Only poll quota when the Quota tab is visible (use Zustand activeTab to check)

• Add virtual scrolling for columns with 30+ cards using @tanstack/react-virtual

• Add Skeleton loading states for async content (card list, chat messages, quota table)

• Add proper TypeScript strict mode validation - tsc --noEmit must pass

### 11.2 Mistakes to Avoid

~~DO NOT: Do NOT poll quota globally when the Quota tab is not visible.~~
~~DO NOT: Do NOT poll pipeline state for cards with finished pipelines.~~
~~DO NOT: Do NOT allow unlimited toast accumulation. Max 5 is already set in Chunk 0 but verify it works.~~

### 11.3 How to Test

• Open DevTools Network tab - verify no unnecessary API calls when tabs are not visible

• Add 35+ cards to a column - verify virtual scrolling works (no lag)

• tsc --noEmit passes with zero errors

## 12. Chunk 9: Responsive Polish

### 12.1 What to Create

• Responsive breakpoints: 320px (mobile - sidebar is full-screen overlay, board scrolls horizontally), 768px (tablet - sidebar collapsed by default), 1024px (desktop - sidebar open by default), 1440px (wide - board columns get more space)

• ARIA labels on all interactive elements

• Keyboard navigation: Tab through cards, Enter to open drawer, Escape to close

• Reduced motion: respect prefers-reduced-motion CSS media query

• Empty state illustrations for columns with no cards

• Loading skeletons for all async data

### 12.2 How to Test

• Resize browser to 375px (mobile) - board scrolls horizontally, sidebar is overlay

• Resize to 768px - sidebar collapses, can be toggled open

• Resize to 1440px - all columns visible without scrolling

• Tab through all interactive elements with keyboard only

• Lighthouse accessibility score >= 90

## 13. Chunk 10: Final Verification and Old Code Deletion

This is the final chunk. Only execute it after ALL previous chunks are verified working.

### 13.1 Final Verification Checklist

• Start backend: cd apps/api && python -m main (runs on port 3000)

• Start new frontend: cd apps/web-v2 && npm run dev (runs on port 5173)

• Test full pipeline: Discover topic -> Topic appears in Column 1 -> Moves to Column 2 -> Save topic -> Start production -> Watch progress in CardDrawer -> Approve in Column 5 -> Card moves to Column 6

• Test chat: Send message -> tokens stream -> tool indicators show -> response completes

• Test all sidebar tabs: YouTube (competitors + own channel), Quota (live data), Models, Skills, KB

• Test drag-and-drop: Valid moves succeed, invalid moves show toast, optimistic updates work

- Test Realtime: Open two browser tabs, changes in one tab reflect in the other
- Test responsive: Works at 375px, 768px, 1024px, 1440px
- Test error handling: Kill backend -> error boundary shows, restart -> recovery works
- Run tsc --noEmit - zero errors
- Run npm run build - successful production build

## 13.2 Delete Old Code

ONLY execute these steps after the verification checklist above passes completely:

- Step 1: Delete the old frontend directory
  - rm -rf apps/web/src/
  - rm apps/web/package.json apps/web/package-lock.json
  - rm apps/web/vite.config.ts apps/web/tsconfig.json
  - rm apps/web/.env apps/web/.env.example
- Step 2: Move new frontend to the standard location
  - mv apps/web-v2/* apps/web/
  - mv apps/web-v2/.* apps/web/ 2>/dev/null
  - rmdir apps/web-v2
- Step 3: Verify the moved build still works
  - cd apps/web && npm install
  - npm run dev - confirm it starts
  - npm run build - confirm production build succeeds
- Step 4: Commit
  - git add apps/web/
  - git commit -m "feat: complete frontend recreation v2.0 - replace old codebase"

## 13.3 What Gets Deleted

| Path | Contents |
| --- | --- |
| apps/web/src/main.tsx | Entry point |
| apps/web/src/App.tsx | Root component |
| apps/web/src/index.css | Styles |
| apps/web/src/types/ | 4 type files |
| apps/web/src/lib/ | 7 library files |
| apps/web/src/hooks/ | 8 hook files |
| apps/web/src/layout/ | 3 layout files |
| apps/web/src/components/ | 22 component files across 7 subdirectories |
| apps/web/package.json | Dependencies |
| apps/web/vite.config.ts | Build config |

Table 4. Old Files to Delete (48 files total)

# 14. Quick Reference: Chunk Dependency Map

| # | Name | Depends On | What You See |
|---|---|---|---|
| 0 | Project Foundation | None | Setup, no UI |
| 1 | Types + API Client | Chunk 0 | No UI, all types |
| 2 | Layout Shell | Chunk 0, 1 | Header + Sidebar visible |
| 3 | Kanban Board | Chunk 1, 2 | Board + Cards + DnD |
| 4 | Card Drawer | Chunk 3 | Drawer + Agent Log |
| 5 | Review Flow | Chunk 4 | Approve/Reject in Drawer |
| 6 | Chat Panel | Chunk 1, 2 | SSE streaming chat |
| 7 | Sidebar Panels | Chunk 2 | YouTube, Quota, etc. |
| 8 | Optimizations | Chunk 1-7 | Performance tuning |
| 9 | Responsive | Chunk 1-8 | Mobile/tablet polish |
| 10 | Cleanup | Chunk 0-9 | Delete old, verify |

Table 5. Chunk Build Order

# 15. Risk Notes

The following risks should be monitored throughout the build process:

- @dnd-kit/react is new and may have breaking changes. Pin the exact version (^1.0.0) and do not upgrade mid-build.
- Supabase schema may change during the build. Use Zod schemas to validate API responses at runtime. If a field is missing or wrong type, the Zod parse will catch it.
- The backend API may not be 100% stable. Test each endpoint call individually before wiring it into a component.
- React 19 ecosystem is still maturing. If a critical package does not support React 19, fall back to React 18 (Chunk 0 decision).